



वेलकम टू द न्यू वीडियो एंड न्यू...
TXT 380.75KB

make notes for me out of this transcript , explain everything properly
deeply

NLP with Machine Learning — Complete Notes

Based on the full transcript. Covers: What is NLP, applications, approaches, pipeline, text preprocessing, feature extraction (One-Hot, Bag of Words, N-Grams, TF-IDF), and the final Emotion Detection project.

1. What is NLP?

NLP = Natural Language Processing

It is a part of **Artificial Intelligence** that helps computers **understand, read, write, and talk like humans**.

- Computers originally understood only **0s and 1s** (binary).
- As many human languages emerged (Hindi, English, Urdu, French, Tamil, etc.), processing all of them became very tough.
- So **NLP** was invented to bridge the gap between **human language** and **computer language**.

In short: NLP allows machines to process human language (English, Hindi, etc.) instead of just 0s and 1s.

Real-life Examples

- Typing "**weather in Delhi**" on Google → Google understands and shows correct weather.
- Saying "**Alexa, play Arijit Singh songs**" → Alexa understands the sentence and plays music.
- **ChatGPT, Gemini, Claude, Anthropic** — all exist because of NLP.

⚠️ Note: Python, Java, C are **computer languages**. Hindi, English, French are **real/natural languages**. NLP deals with natural languages.

2. Real-World Applications of NLP

A. Chatbots & Virtual Assistants

- Examples: ChatGPT, Alexa, Siri, Google Assistant
- NLP allows these tools to:
 - Understand what you're saying/typing
 - Recognize your **intent**
 - Give a relevant **response**
- Involves: **Speech recognition, text understanding, response generation**

B. Email Spam Filtering

- Gmail scans email content for suspicious/spammy phrases like **"You won a lottery"**
- Helps filter spam and send genuine mails to inbox.
- → We will build a **spam filter project** in this course.

C. Sentiment Analysis

- Used on **Twitter, Amazon reviews, customer feedback**
- Example: If a seller sends a ₹10 product instead of ₹1000, people give 1-star reviews. Amazon analyzes this sentiment and can **discard the product**.
- Companies detect whether people feel **positive, negative, or neutral** about their product.
- Useful for **brand monitoring** and **public opinion analysis**.

D. More Applications

- The list keeps growing: AI agents, chatbots, etc.
- NLP is an **evergreen field** — it will never end.
- **You must know NLP before going into AI.**

3. Three Approaches to NLP

Approach 1: Rule-Based Approach (Old School NLP)

- Uses **hand-written rules and grammar logic**
- Basically **if-else statements**
- Example rule: If text contains "lottery" → mark as spam
- Rules define grammar, spelling corrections, sentence structures

Used in: Grammar checkers (Grammarly), spell checkers, simple keyword-based chatbots

Limitations:

- ❌ Can't handle **complex, ambiguous, or new patterns**
- ❌ **Not scalable** for large data
- ❌ Can't understand **sarcasm**
- Example: *"Staying up all night debugging is exactly what I dreamed of"* — sounds positive but is actually sarcasm.

Approach 2: Machine Learning Approach (Statistical Approach)

- Language is treated as **data**
- Algorithms **learn patterns** from data
- Models used: **Naive Bayes, Logistic Regression, Support Vector Machines (SVM)**
- Requires text to be converted into **numbers** using techniques like:
 - **Bag of Words**
 - **TF-IDF**

- Example: Train a model on 10,000 product reviews → it learns which words indicate positive/negative sentiment → predicts for new reviews.
- **Used for:** Sentiment analysis, spam detection, text classification

Limitations:

- ❌ Loses **context** of words
- ❌ Can't understand **sarcasm**
- ❌ Can't understand **sequence of words**
- Example: *"That's what was really helpful"* (said sarcastically after a mistake) → ML model might interpret it as positive feedback.
- Example: *"I'm fine"* (said sadly) → ML can't detect the hidden emotion.

Approach 3: Deep Learning Approach (Modern NLP)

- Came after **2010** when computational power increased
- Uses **Neural Networks** to automatically learn complex patterns and context
- Models can understand: **meaning, context, word order, and more**

Popular Architectures:

- **RNN** (Recurrent Neural Networks) — good for text
- **LSTM** (Long Short-Term Memory) — remembers context across paragraphs
- **GRU** (Gated Recurrent Units)
- **Transformers** — GPT, BERT, T5

How it converts text to numbers:

- Doesn't use traditional Bag of Words / TF-IDF
- Uses **Word Embeddings**:
 - **Word2Vec**
 - **GloVe**
 - **FastText**
- Creates **dense vectors** that understand relationships and context.

🔪 This video focuses on **NLP with Machine Learning**. Deep Learning NLP will be covered later.

4. NLP Pipeline (Step-by-Step)**Step 1: Gathering the Text/Data**

Ways to collect data:

Method	Description
1. Prebuilt Public Datasets	Kaggle, etc. Example: Sentiment Analysis dataset (474 KB), Twitter Sentiment Analysis (2 MB, 69,491 tweets)
2. Web Scraping	When data isn't readily available. Use libraries like BeautifulSoup . Scrapes HTML tags (<code><p></code> , <code></code> , <code></code>) — these need cleaning later.

Method	Description
3. APIs	Structured + legal way to get live data. Twitter API, LinkedIn API, Instagram API.
4. Manual Collection / Crowd Sourcing	Example: Google Form shared with students to write reviews about "Sheriyans" → got 1000+ reviews.
5. Auto-Generate Data	Text augmentation using NLP tools/libraries.

Step 2: Text Cleaning (Preprocessing)

This is one of the MOST IMPORTANT steps — done in both ML and DL.

Step	What it does	Why
Lower Casing	Convert all text to lowercase	"The" and "the" mean the same but ML interprets them differently
Remove Punctuations	Remove ? ! @ # \$ % ^ & *	They don't add meaning in ML context. ⚠ In DL (BERT, GPT, T5), punctuation can carry meaning and is usually kept.
Remove Numbers	Remove digits	Optional — depends on task. "I bought 5 iPhones" — may not need the number.
Remove URLs & Links	Remove https://...	Very common in social media data
Remove HTML Tags	Remove <p> , , etc.	From web scraping — they add no meaning
Remove Emojis & Special Characters	Remove emojis	Emojis might break vectorization if not removed/handled
Remove Stopwords	Remove common words: is, the, was, and, in, for	They don't help understand meaning. ⚠ In DL, keep them if you want deeper meaning.
Spelling Correction	Fix spelling mistakes	Using TextBlob or other libraries (optional)

5. Key Terminology

Term	Meaning
Corpus	Collection of all text (e.g., all 10,000/16,000 rows together)
Document	A single piece of text (one row/sentence/review)
Vocabulary	List of all unique words in the corpus

Term	Meaning
Tokens	Individual words/symbols after splitting (e.g., "I love NLP" → "I", "love", "NLP")
Tokenization	Process of splitting text into tokens
Vectorization	Converting text into numbers/vectors

6. Feature Extraction / Vectorization Techniques

Why? ML models only understand **numbers**, not words, feelings, or emojis. So we convert text → numbers in a smart way.

Technique 1: One-Hot Encoding

How it works:

- Create a vocabulary of all unique words
- Each word gets a vector of size = vocabulary size
- The word's position = 1, all others = 0

Example:

text

Corpus:

D1: Arsh Wash Sheriyans

D2: Harsh Also Wash Sheriyans

D3: Sheriyans Teach Arsh

Vocabulary: [Arsh, Wash, Sheriyans, Harsh, Also, Teach] (size = 6)

D1 vector:

Arsh → [1,0,0,0,0,0]

Wash → [0,1,0,0,0,0]

Sheriyans → [0,0,1,0,0,0]

Shape: 3×6

D2 vector:

Harsh → [0,0,0,1,0,0]

Also → [0,0,0,0,1,0]

Wash → [0,1,0,0,0,0]

Sheriyans → [0,0,1,0,0,0]

Shape: 4×6

Pros:

-  Intuitive and easy to understand
-  Easy to implement

Cons:

-  **Sparsity** — too many zeros
-  **Out of Vocabulary (OOV)** — new words get all zeros
-  **Size difference** — different sentences have different vector sizes (ML needs fixed size)
-  **Semantic meaning not captured** — "time" and "success" are related, but "iPad" and "time" are not. One-hot gives them all the same distance.

⚠️ **One-Hot Encoding is the least used algorithm** — almost never used in modern NLP. But it's important to understand because Bag of Words builds on it.

Technique 2: Bag of Words (BoW)

What it is:

- Converts text into numbers by **counting word frequency**
- Creates a list of all unique words (vocabulary)
- For each sentence, counts how many times each word appears
- **Doesn't care about grammar or word order**
- Called "Bag" because it's a **jumbled, unordered collection** of words

Example:

text

Corpus:

D1: Arsh Wash Sheriyans

D2: Harsh Also Wash Sheriyans

D3: Sheriyans Teach Sheriyans (Sheriyans repeated twice)

Vocabulary: [Arsh, Wash, Sheriyans, Harsh, Also, Teach]

D1: [1, 1, 1, 0, 0, 0]

D2: [0, 1, 1, 1, 1, 0]

D3: [0, 0, 2, 0, 0, 1] ← Sheriyans counted twice

Why frequency matters:

- Words appearing the same number of times are "similar"
- Words appearing different times are "different"
- This captures **some semantic meaning** (better than one-hot)

Implementation (sklearn):

python

```
from sklearn.feature_extraction.text import CountVectorizer

vectorizer = CountVectorizer(
    max_features=None,      # None = use all words; 1 = only most frequent word
    vocabulary=None,        # None = auto-create vocabulary
    binary=True,            # True = only 0/1 (presence), False = actual counts
    ngram_range=(1,1)      # (1,1) = unigrams; (1,2) = unigrams + bigrams
)

X = vectorizer.fit_transform(documents)
print(vectorizer.get_feature_names_out())
```

Parameters explained:

Parameter	Meaning
max_features	Limit vocabulary to top N most frequent words. None = all words
vocabulary	Pass your own vocabulary or let it auto-create
binary	True = only 0/1; False = actual counts

Parameter	Meaning
ngram_range	(1,1) = single words; (1,2) = unigrams + bigrams; (2,2) = only bigrams

Pros:

-  Easy to implement
-  **Fixed size** — works with ML models
-  Works with ML models
-  Fast and efficient (better than one-hot)

Cons:

-  **Sparsity** — still many zeros for large vocabularies
-  **Out of Vocabulary (OOV)** — new words get all zeros
-  **Semantic meaning still not fully captured** (better than one-hot, but not as good as DL)
-  **Out of order** — word order is lost

Technique 3: N-Grams (Unigrams, Bigrams, Trigrams, etc.)**What it is:**

- Instead of single words, use **groups of N consecutive words**
- **Unigram** = 1 word
- **Bigram** = 2 consecutive words
- **Trigram** = 3 consecutive words
- **N-gram** = N consecutive words

Example:

text

Corpus:

D1: Arsh Wash Sheriyans

D2: Harsh Also Wash Sheriyans

D3: Sheriyans Teach Sheriyans

Unigram Vocabulary:

[Arsh, Wash, Sheriyans, Harsh, Also, Teach]

Bigram Vocabulary:

[Arsh Wash, Wash Sheriyans, Harsh Also, Also Wash, Wash Sheriyans (dup), Sheriyans Teach, Teach Sheriyans]

Trigram Vocabulary:

[Arsh Wash Sheriyans, Harsh Also Wash, Also Wash Sheriyans, Sheriyans Teach Sheriyans]

Why N-grams matter — Semantic Meaning Example:

Consider:

- Sentence 1: "Cricket is **very** good"
- Sentence 2: "Cricket is **not** good"

With **unigrams** (Bag of Words):

text

Vocabulary: [Cricket, is, very, good, not]
 D1: [1, 1, 1, 1, 0]
 D2: [1, 1, 0, 1, 1]
 → Only 2 dimensions differ (very vs not)
 → Sentences look very similar

With bigrams:

text

Vocabulary: [Cricket is, is very, very good, is not, not good]
 D1: [1, 1, 1, 0, 0]
 D2: [1, 0, 0, 1, 1]
 → 4 dimensions differ
 → Sentences look very different ✓

Implementation:

```
python

# Unigrams only
vectorizer = CountVectorizer(ngram_range=(1,1))

# Unigrams + Bigrams
vectorizer = CountVectorizer(ngram_range=(1,2))

# Only Bigrams
vectorizer = CountVectorizer(ngram_range=(2,2))

# Only Trigrams
vectorizer = CountVectorizer(ngram_range=(3,3))
```

Pros:

- ✓ Better semantic meaning captured (compared to BoW)
- ✓ Helps differentiate similar sentences

Cons:

- ✗ Dimensionality increases significantly
- ✗ Out of Vocabulary (OOV) still a problem
- ✗ Still suffers from sparsity

💡 Bag of N-Grams = Bag of Words + N-grams

Technique 4: TF-IDF (Term Frequency — Inverse Document Frequency)

The Problem with BoW:

- Gives **same weight** to all words
- Common words like "is", "the" get high counts but no meaning
- Rare/important words don't get enough weight

TF-IDF solves this by assigning weights.

Term Frequency (TF)

Formula:

text

$TF(t, d) = (\text{Number of times term } t \text{ appears in document } d) / (\text{Total number of terms in document } d)$

Example:

text

D1: Arsh Wash Sheriyans (3 terms)

$TF(\text{Arsh}) = 1/3$

$TF(\text{Wash}) = 1/3$

$TF(\text{Sheriyans}) = 1/3$

TF is always between 0 and 1 (usually).

Inverse Document Frequency (IDF)**Formula:**

text

$IDF(t) = \log(\text{Total number of documents in corpus} / \text{Number of documents containing term } t)$

Example:

text

Corpus: 3 documents (D1, D2, D3)

$IDF(\text{Arsh}) = \log(3/2)$ ← Arsh appears in D1 and D3

$IDF(\text{Sheriyans}) = \log(3/3) = \log(1) = 0$ ← appears in all 3 docs

$IDF(\text{Harsh}) = \log(3/1)$ ← appears only in D2

Interpretation:

- **Rare words** → higher IDF → higher weight
- **Common words** (appearing in all documents) → IDF = 0 → no weight

Final TF-IDF Score

text

$TF-IDF(t, d) = TF(t, d) \times IDF(t)$

Example:

text

D1: Arsh Wash Sheriyans

$TF-IDF(\text{Arsh}, D1) = (1/3) \times \log(3/2) \approx 0.33 \times 0.176 \approx 0.058$

$TF-IDF(\text{Wash}, D1) = (1/3) \times \log(3/2) \approx 0.058$

$TF-IDF(\text{Sheriyans}, D1) = (1/3) \times \log(3/3) = (1/3) \times 0 = 0$

Implementation:

python

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(documents)
```

Pros:

-  **Information retrieval** — great for Google Search, etc.
-  Gives **different weights** to words based on importance
-  Better than BoW for many tasks

Cons:

-  **Sparsity** still exists
-  **Out of Vocabulary (OOV)** still a problem
-  **Dimensionality** can be huge (e.g., 50,000 dimensions)
-  Still doesn't capture deep semantic meaning like DL embeddings

7. Summary of Vectorization Techniques

Technique	How it works	Pros	Cons
One-Hot Encoding	Each word = vector with 1 at its position, 0 elsewhere	Intuitive, easy	Sparsity, OOV, size, no semantic meaning
Bag of Words	Count word frequency per document	Easy, fixed size, works with ML	Sparsity, OOV, no semantics
N-Grams	Use groups of N consecutive words	Better semantics	Higher dimension
TF-IDF	Weight words by $TF \times IDF$	Information retrieval, weighted	Sparsity, OOV, high semantics

Progression: One-Hot → Bag of Words → N-Grams → TF-IDF → (Deep Learning: Word2Vec, GloVe, BERT, Transformers)

8. The Project: Emotion Detection / Sentiment Analysis

Dataset

- **Source:** Kaggle — "Emotions Dataset for NLP"
- **Files:** train.txt , test.txt , val.txt
- **Format:** text;emotion
- **Example:**

```
text

i didnt feel humiliated;sadness
i can go from feeling so hopeless to so damned hopeful;happiness
im feeling rather rotten;anger
```

Step-by-Step Implementation

Step 1: Import Libraries

```
python

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import string
import nltk
```

Step 2: Load Data

```
python

df = pd.read_csv('train.txt', sep=';', header=None, names=['text', 'emotion'])
```

Step 3: Check for Null Values

```
python

df.isnull().sum()  # → 0 null values
```

Step 4: Convert Emotions to Numbers

```
python

# Get unique emotions
unique_emotions = df['emotion'].unique()
# → ['sadness', 'anger', 'love', 'surprise', 'fear', 'joy']

# Create mapping dictionary
emotion_numbers = {}
i = 0
for emotion in unique_emotions:
    emotion_numbers[emotion] = i
    i += 1

# Apply mapping
df['emotion'] = df['emotion'].map(emotion_numbers)
```

Step 5: Text Preprocessing**a) Lower Casing:**

```
python

df['text'] = df['text'].apply(lambda x: x.lower())
```

b) Remove Punctuations:

```
python

def remove_punctuation(text):
    return text.translate(str.maketrans('', '', string.punctuation))

df['text'] = df['text'].apply(remove_punctuation)
```

c) Remove Numbers:

```
python

def remove_numbers(text):
    new_text = ""
```

```

for i in text:
    if not i.isdigit():
        new_text += i
return new_text

df['text'] = df['text'].apply(remove_numbers)

```

d) Remove URLs & Links: (similar function approach)

e) Remove HTML Tags: (similar function approach)

f) Remove Emojis & Special Characters:

```

python

def remove_emojis(text):
    new_text = ""
    for i in text:
        if i.isascii(): # Keep only ASCII characters
            new_text += i
    return new_text

df['text'] = df['text'].apply(remove_emojis)

```

ASCII values: A=65, B=66, a=97, b=98, etc. Emojis have Unicode values, not ASCII.

g) Remove Stopwords:

```

python

from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

nltk.download('punkt')
nltk.download('stopwords')

stop_words = set(stopwords.words('english')) # 198 stopwords

def remove_stopwords(text):
    words = text.split() # or word_tokenize(text)
    clean_text = []
    for word in words:
        if word not in stop_words:
            clean_text.append(word)
    return " ".join(clean_text)

df['text'] = df['text'].apply(remove_stopwords)

```

Example:

text

Before: "i can go from feeling so hopeless to so damned hopeful"

After: "feeling hopeless damned hopeful"

Step 6: Train-Test Split

```

python

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    df['text'], df['emotion'], test_size=0.2, random_state=42
)

# 80% train, 20% test

```

Step 7: Vectorization

Bag of Words:

```
python

from sklearn.feature_extraction.text import CountVectorizer
from sklearn.feature_extraction.text import TfidfVectorizer

bow_vectorizer = CountVectorizer()
X_train_bow = bow_vectorizer.fit_transform(X_train)
X_test_bow = bow_vectorizer.transform(X_test)

tfidf_vectorizer = TfidfVectorizer()
X_train_tfidf = tfidf_vectorizer.fit_transform(X_train)
X_test_tfidf = tfidf_vectorizer.transform(X_test)
```

Step 8: Model Training

Model 1: Naive Bayes (Multinomial)

```
python

from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score

nb_model = MultinomialNB()
nb_model.fit(X_train_bow, y_train)

y_pred = nb_model.predict(X_test_bow)
print(accuracy_score(y_test, y_pred))
# → ~76% accuracy
```

Why Multinomial Naive Bayes? It's specifically suited for **discrete features** like word counts and frequencies in text classification. Gaussian NB is for normal distribution (numeric data).

Model 2: Naive Bayes with TF-IDF

```
python

nb2_model = MultinomialNB()
nb2_model.fit(X_train_tfidf, y_train)

y_pred2 = nb2_model.predict(X_test_tfidf)
print(accuracy_score(y_test, y_pred2))
# → ~66% accuracy
```

Model 3: Logistic Regression with TF-IDF

```
python

from sklearn.linear_model import LogisticRegression

log_model = LogisticRegression(max_iter=1000)
log_model.fit(X_train_tfidf, y_train)

log_pred = log_model.predict(X_test_tfidf)
print(accuracy_score(y_test, log_pred))
# → ~86% accuracy 🟢 BEST
```

Why max_iter=1000? Default is 100. For large text data, the solver may not converge in 100 iterations, so we increase it.

Results Summary

Model	Vectorizer	Accuracy
Multinomial Naive Bayes	Bag of Words	~76%
Multinomial Naive Bayes	TF-IDF	~66%
Logistic Regression	TF-IDF	~86% 

Next Steps (Homework)

1. Try **multiple models** (SVM, etc.)
2. **Hyperparameter tuning** (tune CountVectorizer, TfidfVectorizer parameters)
3. Build a **UI using Streamlit**
4. Upload project on **LinkedIn** and tag the instructor

9. Key Takeaways

1. **NLP** bridges human language and computers.
2. **Three approaches:** Rule-Based → ML-Based → DL-Based.
3. **Pipeline:** Data Gathering → Text Cleaning → Feature Extraction → Model Training → Evaluation.
4. **Text Cleaning is crucial** — lower casing, removing punctuation, numbers, URLs, HTML tags, emojis, stopwords.
5. **Vectorization techniques:** One-Hot → BoW → N-Grams → TF-IDF.
6. **TF-IDF** gives weights based on importance (rare words = higher weight).
7. **N-Grams** help capture semantic meaning (e.g., "very good" vs "not good").
8. **Best model** in project: **Logistic Regression + TF-IDF (86% accuracy)**.
9. **Deep Learning NLP** (Word2Vec, GloVe, BERT, Transformers) will be covered later.
10. **You must know NLP before going into AI.**

10. Important Terms Glossary

Term	Definition
NLP	Natural Language Processing
Corpus	Collection of all text documents
Document	Single piece of text (one row)
Vocabulary	Set of all unique words in corpus
Token	Individual word/symbol
Tokenization	Splitting text into tokens
Vectorization	Converting text to numbers
Stopwords	Common words with little meaning (is, the, and)

Term	Definition
Sparsity	Too many zeros in a matrix
OOV	Out of Vocabulary — new words not in vocabulary
TF	Term Frequency
IDF	Inverse Document Frequency
N-Gram	Group of N consecutive words
Bigram	2 consecutive words
Trigram	3 consecutive words
BoW	Bag of Words
ASCII	American Standard Code for Information Interchange
Unicode	Universal code for all characters/emojis

11. Homework / Practice Tasks

1.  Complete the **Emotion Detection** project with 86%+ accuracy.
2.  Try **SVM** and other models.
3.  Do **hyperparameter tuning** on vectorizers.
4.  Build a **Streamlit** UI for the project.
5.  Upload on **LinkedIn** and tag the instructor.
6.  Comment "**Till the end**" if you completed the video.

 **Next Video:** Deep Learning → Then NLP with Deep Learning → Then Advanced Projects.

End of Notes 

give me interview questions with answers topic wise of all the things covered in this chat

NLP with Machine Learning — Interview Questions & Answers

Topic-wise interview preparation based on the complete transcript.

Topic 1: Fundamentals of NLP

Q1. What is NLP? Why do we need it?

Answer:

NLP (Natural Language Processing) is a subfield of Artificial Intelligence that enables computers to **understand, interpret, read, write, and communicate** using human language.

Why we need it:

- Computers originally understood only **binary (0s and 1s)**.
- As human languages (Hindi, English, French, Tamil, etc.) evolved, processing all of them became very difficult.
- NLP bridges the gap between **human language** and **computer language**, allowing machines to process and understand natural language instead of just binary.

Real-world examples: Google Search ("weather in Delhi"), Alexa ("play Arijit Singh songs"), ChatGPT, Gemini, Siri, Google Assistant.

Q2. Differentiate between Natural Language and Computer Language.**Answer:**

Natural Language	Computer Language
Hindi, English, French, Tamil, Urdu	Python, Java, C, C++
Used by humans for communication	Used to instruct computers
Ambiguous, context-dependent	Strict syntax, unambiguous
Processed using NLP	Processed using compilers/interpreters

Q3. What are the main real-world applications of NLP?**Answer:**

1. **Chatbots & Virtual Assistants** — ChatGPT, Alexa, Siri, Google Assistant
2. **Email Spam Filtering** — Gmail detecting "You won a lottery" as spam
3. **Sentiment Analysis** — Twitter analysis, Amazon reviews, customer feedback
4. **Machine Translation** — Google Translate
5. **Speech Recognition** — Voice-to-text
6. **Text Summarization** — News summarization
7. **Question Answering** — Google Search, chatbots
8. **Named Entity Recognition (NER)** — Extracting names, places, organizations
9. **AI Agents** — Modern AI-based automation

 Topic 2: Approaches to NLP**Q4. What are the three main approaches to NLP? Compare them.****Answer:**

Approach	Era	How it works	Pros	Cons
Rule-Based (Old School)	Pre-1990s	Hand-written rules + grammar logic (if-else)	Simple, interpretable	Not scalable, ambiguous

Approach	Era	How it works	Pros	Cons
Machine Learning (Statistical)	1990s–2010	Treats language as data, learns patterns using Naive Bayes, Logistic Regression, SVM	Scalable, works with ML models	Loses context, struggles with sarcasm
Deep Learning (Modern)	2010+	Neural networks (RNN, LSTM, GRU, Transformers) learn complex patterns	Captures context, semantics, word order	Needs computational power

Q5. Explain Rule-Based NLP with an example and its limitations.

Answer:
 Rule-Based NLP uses **hand-written rules and grammar logic** — essentially if-else statements.

Example rule:

```

text

If text contains "lottery" → mark as spam
If sentence contains "not good" → mark as negative
  
```

Used in: Grammar checkers (Grammarly), spell checkers, simple keyword-based chatbots.

- Limitations:**
- ✗ Cannot handle **complex, ambiguous, or new patterns**
 - ✗ **Not scalable** for large data
 - ✗ Cannot understand **sarcasm**
 - Example: *"Staying up all night debugging is exactly what I dreamed of"* — sounds positive but is sarcasm.
 - ✗ Handwriting thousands of rules is impractical.

Q6. Why did Machine Learning become preferred over Rule-Based NLP?

Answer:
 Because ML:

- Treats language as **data** and learns patterns automatically
- Scales** to large datasets
- Doesn't require manually writing thousands of rules
- Works well for **sentiment analysis, spam detection, text classification**

Models used: Naive Bayes, Logistic Regression, SVM.

- But limitations remain:**
- Loses **context**
 - Can't detect **sarcasm**
 - Ignores **word order**

- Example: "I'm fine" (said sadly) → ML can't detect hidden emotion.

Q7. What is the Deep Learning approach to NLP? Which architectures are used?

Answer:

Deep Learning NLP uses **neural networks** to automatically learn complex patterns and context from text. It emerged after 2010 due to increased computational power.

Popular architectures:

- **RNN (Recurrent Neural Networks)** — for sequential text
- **LSTM (Long Short-Term Memory)** — remembers context across long paragraphs
- **GRU (Gated Recurrent Units)** — lighter than LSTM
- **Transformers** — GPT, BERT, T5

Key difference from ML: Instead of Bag of Words / TF-IDF, DL uses **Word Embeddings** (Word2Vec, GloVe, FastText) which create **dense vectors** that capture semantic meaning and context.

Topic 3: NLP Pipeline

Q8. Explain the NLP pipeline step by step.

Answer:

Step 1: Data Gathering

- Prebuilt public datasets (Kaggle)
- Web scraping (BeautifulSoup)
- APIs (Twitter, LinkedIn, Instagram)
- Manual collection / crowd sourcing
- Auto-generation / text augmentation

Step 2: Text Cleaning (Preprocessing)

- Lower casing
- Remove punctuations
- Remove numbers
- Remove URLs/links
- Remove HTML tags
- Remove emojis/special characters
- Remove stopwords
- Spelling correction (optional)

Step 3: Feature Extraction / Vectorization

- One-Hot Encoding
- Bag of Words
- N-Grams
- TF-IDF

- (DL: Word2Vec, GloVe, BERT)

Step 4: Model Training

- Naive Bayes, Logistic Regression, SVM, etc.

Step 5: Evaluation

- Accuracy score, classification report, confusion matrix

Step 6: Deployment

- Streamlit UI, Flask API, etc.

Q9. What are the different ways to gather data for an NLP project?

Answer:

1. **Prebuilt Public Datasets** — Kaggle (e.g., Sentiment Analysis dataset, Twitter Sentiment Analysis)
2. **Web Scraping** — When data isn't readily available; use BeautifulSoup
3. **APIs** — Twitter API, LinkedIn API, Instagram API (structured + legal)
4. **Manual Collection / Crowd Sourcing** — Google Forms, surveys
5. **Auto-Generation / Text Augmentation** — Using NLP tools/libraries

📌 Topic 4: Text Preprocessing

Q10. Why is text cleaning important in NLP?

Answer:

Text cleaning is important because:

- Raw text contains **noise** (punctuation, HTML tags, URLs, emojis, numbers)
- ML models only understand **numbers**, not raw text
- Noise can **break vectorization**
- Common words (stopwords) add **no meaning** but increase dimensionality
- Case differences ("The" vs "the") confuse ML models

It is done in both ML and DL pipelines (though DL keeps punctuation and stopwords sometimes).

Q11. What is lower casing and why do we do it?

Answer:

Lower casing means converting all text to lowercase.

Why:

- "The" and "the" mean the same, but ML interprets them as different words
- Reduces vocabulary size
- Avoids duplicate features

Example: "I love NLP" → "i love nlp"

Q12. Why do we remove punctuation in ML-based NLP but keep them in DL-based NLP?

Answer:

ML-Based NLP	DL-Based NLP
Punctuation adds no meaning	Punctuation can carry meaning
Removed during preprocessing	Usually kept during preprocessing
Example: "good!" and "good" treated same	Example: "good!" vs "good?" have different tones
Simpler models (BoW, TF-IDF)	Models like BERT, GPT, T5 use punctuation for context

Q13. What are stopwords? Why remove them?

Answer:

Stopwords are **common words** that appear frequently but add little meaning — e.g., *is, the, and, in, for, was, a, an*.

Why remove:

- They add **noise**
- They increase **dimensionality**
- They don't help in understanding meaning

Example:

text

Before: "Bhopal is a peaceful place for you"

After: "Bhopal peaceful place"

Note: In DL, stopwords may be kept if you want deeper meaning.

NLTK has 198 English stopwords.

Q14. Why do we remove emojis?

Answer:

- Emojis can **break vectorization** if not handled properly
- They have **Unicode values**, not ASCII
- Many different emojis exist, causing vocabulary explosion
- Unless the task specifically requires emoji sentiment (e.g., Twitter sentiment), they are removed

Method: Keep only ASCII characters (`i.isascii()`).

Q15. What is the difference between ASCII and Unicode?

Answer:

ASCII	Unicode
American Standard Code for Information Interchange	Universal code for all characters
128 characters (letters, digits, symbols)	1,00,000+ characters (all languages, emojis)
A=65, a=97, B=66	Emojis, Hindi, Arabic, etc.
Limited	Universal

In NLP: We often keep only ASCII characters to remove emojis and special characters.

 Topic 5: Key Terminology

Q16. Define: Corpus, Document, Vocabulary, Token, Tokenization.

Answer:

Term	Definition	Example
Corpus	Collection of all text documents	All 10,000 reviews together
Document	A single piece of text	One review: "I love this product"
Vocabulary	Set of all unique words in corpus	[I, love, this, product]
Token	Individual word/symbol	"I", "love", "this", "product"
Tokenization	Process of splitting text into tokens	"I love NLP" → ["I", "love", "NLP"]

Q17. What is the difference between Tokenization and Splitting?

Answer:

Splitting	Tokenization
Splits on spaces only	Splits on spaces + punctuation
"I am happy!" → ["I", "am", "happy!"]	"I am happy!" → ["I", "am", "happy", "!"]
Simple <code>.split()</code>	Uses NLTK <code>word_tokenize</code>
Doesn't separate punctuation	Separates punctuation as tokens

Tokenization is better because it handles punctuation properly.

 Topic 6: Feature Extraction / Vectorization

Q18. What is Feature Extraction / Vectorization? Why is it needed?

Answer:

Feature Extraction (or Vectorization) is the process of **converting text into numerical**

vectors so that ML models can understand and process them.

Why needed:

- ML models only understand **numbers**
- They cannot understand words, feelings, or emojis
- Text must be converted into a **smart numerical representation**

Techniques:

- One-Hot Encoding
- Bag of Words (BoW)
- N-Grams
- TF-IDF
- (DL: Word2Vec, GloVe, BERT)

Q19. Explain One-Hot Encoding with an example.

Answer:

One-Hot Encoding creates a vector for each word where:

- Vector size = vocabulary size
- The word's position = 1
- All other positions = 0

Example:

text

Corpus:

D1: Arsh Wash Sheriyans

D2: Harsh Also Wash Sheriyans

D3: Sheriyans Teach Arsh

Vocabulary: [Arsh, Wash, Sheriyans, Harsh, Also, Teach] (size=6)

D1:

Arsh → [1,0,0,0,0,0]

Wash → [0,1,0,0,0,0]

Sheriyans → [0,0,1,0,0,0]

Shape: 3×6

Pros: Intuitive, easy to implement

Cons: Sparsity, OOV, size difference, no semantic meaning

Least used algorithm in modern NLP — but important for understanding BoW.

Q20. What is Bag of Words (BoW)? How does it work?

Answer:

Bag of Words converts text into numbers by **counting word frequency** per document.

Steps:

1. Create vocabulary of all unique words
2. For each document, count how many times each word appears

3. Create a vector of counts

Example:

text

D1: Arsh Wash Sheriyans → [1,1,1,0,0,0]

D2: Harsh Also Wash Sheriyans → [0,1,1,1,1,0]

D3: Sheriyans Teach Sheriyans → [0,0,2,0,0,1]

Called "**Bag**" because it's an **unordered collection** of words — grammar and order don't matter.

Pros: Easy, fixed size, works with ML

Cons: Sparsity, OOV, no order, limited semantics

Q21. What is the difference between One-Hot Encoding and Bag of Words?

Answer:

One-Hot Encoding	Bag of Words
Each word = separate vector	Each document = single vector
Binary (0 or 1)	Counts (0, 1, 2, 3...)
Size = vocabulary size × words	Size = vocabulary size
No frequency info	Frequency info included
Rarely used	Commonly used
Doesn't work with ML (size difference)	Works with ML (fixed size)

Q22. What are N-Grams? Explain with an example.

Answer:

N-Grams are **groups of N consecutive words**.

- **Unigram** = 1 word
- **Bigram** = 2 consecutive words
- **Trigram** = 3 consecutive words
- **N-gram** = N consecutive words

Example:

text

Sentence: "Cricket is very good"

Unigrams: [Cricket, is, very, good]

Bigrams: [Cricket is, is very, very good]

Trigrams: [Cricket is very, is very good]

Why use N-Grams?

- Captures **word order** and **context**

- Example: "very good" vs "not good" — bigrams differentiate them better than unigrams

Implementation:

```
python

CountVectorizer(ngram_range=(1,2)) # unigrams + bigrams
CountVectorizer(ngram_range=(2,2)) # only bigrams
```

Cons: Increases dimensionality, OOV still a problem.

Q23. How do N-Grams help capture semantic meaning? Explain with "very good" vs "not good".

Answer:

With Unigrams (BoW):

```
text

Vocabulary: [Cricket, is, very, good, not]
"Cricket is very good" → [1,1,1,1,0]
"Cricket is not good" → [1,1,0,1,1]
→ Only 2 dimensions differ → sentences look similar
```

With Bigrams:

```
text

Vocabulary: [Cricket is, is very, very good, is not, not good]
"Cricket is very good" → [1,1,1,0,0]
"Cricket is not good" → [1,0,0,1,1]
→ 4 dimensions differ → sentences look very different ✓
```

Conclusion: N-Grams capture **local word order** and help differentiate sentences with opposite meanings.

Q24. What is TF-IDF? Explain TF and IDF with formulas.

Answer:

TF-IDF (Term Frequency — Inverse Document Frequency) assigns **weights** to words based on their importance.

Term Frequency (TF):

```
text


$$TF(t, d) = (\text{Number of times term } t \text{ appears in document } d) / (\text{Total terms in document } d)$$

```

- Always between 0 and 1

Inverse Document Frequency (IDF):

```
text


$$IDF(t) = \log(\text{Total documents in corpus} / \text{Number of documents containing term } t)$$

```

- Rare words → higher IDF
- Common words → lower IDF (0 if in all documents)

Final Score:

text

$$TF\text{-}IDF(t, d) = TF(t, d) \times IDF(t)$$

Example:

text

D1: Arsh Wash Sheriyans (3 terms)

$TF(\text{Arsh}) = 1/3$
 $IDF(\text{Arsh}) = \log(3/2)$ [appears in 2 docs]
 $TF\text{-}IDF(\text{Arsh}) = (1/3) \times \log(3/2) \approx 0.058$

$TF(\text{Sheriyans}) = 1/3$
 $IDF(\text{Sheriyans}) = \log(3/3) = 0$ [appears in all docs]
 $TF\text{-}IDF(\text{Sheriyans}) = 0$

Q25. Why is TF-IDF better than Bag of Words?

Answer:

Bag of Words	TF-IDF
Gives same weight to all words	Gives different weights based on importance
Common words get high counts	Common words get low/zero weight
Rare words ignored	Rare words get high weight
No importance differentiation	Importance differentiation

Example: In BoW, "is" and "Sheriyans" both get count 1. In TF-IDF, "Sheriyans" (common) gets weight 0, while "Arsh" (rare) gets positive weight.

Use case: Google Search uses TF-IDF-like ranking for information retrieval.

Q26. Compare all vectorization techniques.

Answer:

Technique	How it works	Pros	Cons
One-Hot	1 at word position, 0 elsewhere	Intuitive	Sparsity, OOV, size is
BoW	Count word frequency	Fixed size, works with ML	Sparsity, OOV, no ord
N-Grams	Groups of N words	Better semantics	High dimensions, OC
TF-IDF	Weight = TF × IDF	Information retrieval, weighted	Sparsity, OOV, high c

Progression: One-Hot → BoW → N-Grams → TF-IDF → (DL: Word2Vec, GloVe, BERT)

Q27. What is Out of Vocabulary (OOV)? Why is it a problem?

Answer:

OOV (Out of Vocabulary) means a word appears in **new/test data** that was **not in the training vocabulary**.

Why it's a problem:

- The model has no vector representation for the new word
- It gets all zeros in the vector
- Model cannot make accurate predictions

Example:

text

Training vocabulary: [Arsh, Wash, Sheriyans, Harsh, Also, Teach]

New sentence: "Sheriyans is cool"

→ "Sheriyans" exists 

→ "is" and "cool" don't exist  → all zeros

Present in: One-Hot, BoW, N-Grams, TF-IDF (all ML-based techniques)

Q28. What is Sparsity? Why is it a problem?

Answer:

Sparsity means having **too many zeros** in the feature matrix.

Why it happens:

- Vocabulary size can be 50,000+
- Each document has only ~50 words
- So 49,950+ positions are zeros

Why it's a problem:

- Wastes memory and computation
- Makes the matrix very large and inefficient
- Slows down training

Present in: One-Hot, BoW, N-Grams, TF-IDF

Q29. What is `max_features` in `CountVectorizer`?

Answer:

`max_features` limits the vocabulary to the **top N most frequent words**.

Example:

python

```
CountVectorizer(max_features=1)
# Only the most frequent word is kept
```

Why use it:

- Reduces dimensionality
- Reduces sparsity
- Speeds up training
- Helps when vocabulary is huge (50,000+)

Default: None (use all words)

Q30. What is `binary=True` in `CountVectorizer`?

Answer:

`binary=True` converts counts to **0 or 1** — only presence/absence, not frequency.

Example:

text

Without binary: [1, 2, 1, 0, 0] (Sheriyans appeared twice)

With binary: [1, 1, 1, 0, 0] (Sheriyans appeared at least once)

Why use it:

- Reduces importance of repeated words
- Sometimes presence matters more than frequency

Topic 7: Model Training & Evaluation

Q31. Which ML models are commonly used for NLP tasks?

Answer:

1. **Naive Bayes** (Multinomial) — best for text classification, word counts
2. **Logistic Regression** — works well with TF-IDF, probability-based
3. **Support Vector Machines (SVM)** — good for high-dimensional data
4. **Random Forest** — ensemble method
5. **Gradient Boosting** — advanced ensemble

Best for NLP: Multinomial Naive Bayes and Logistic Regression.

Q32. Why Multinomial Naive Bayes and not Gaussian Naive Bayes for text?

Answer:

Gaussian NB	Multinomial NB
Assumes normal distribution	Assumes discrete features
Used for numeric/continuous data	Used for word counts/frequencies
Not suitable for text	Specifically suited for text classification

Scikit-learn documentation says: "Multinomial Naive Bayes is a variant of Naive Bayes classifier specifically suited for classification tasks where features are discrete such as

word counts and frequencies in text classification."

Q33. Why did Logistic Regression with TF-IDF perform better than Naive Bayes?

Answer:

Logistic Regression achieved **86% accuracy** vs Naive Bayes' 76% (BoW) and 66% (TF-IDF).

Reasons:

- Logistic Regression works well with **probability-based weights** (TF-IDF gives 0.6, 0.7, etc.)
- TF-IDF values are between 0 and 1, which suits Logistic Regression's log-loss function
- Logistic Regression handles **high-dimensional sparse data** well
- Naive Bayes assumes **feature independence**, which is rarely true in text

Q34. Why `max_iter=1000` in Logistic Regression?

Answer:

- Default `max_iter` is **100**
- For large text data (high-dimensional vectors), the solver may **not converge** in 100 iterations
- Increasing to 1000 ensures convergence
- Prevents "Solver has not converged" warning

Q35. What is the difference between `fit_transform()` and `transform()` ?

Answer:

<code>fit_transform()</code>	<code>transform()</code>
Used on training data	Used on test data
Learns vocabulary + transforms	Only transforms using learned vocabulary
<code>vectorizer.fit_transform(X_train)</code>	<code>vectorizer.transform(X_test)</code>
Creates vocabulary	Uses existing vocabulary

Important: Never use `fit_transform()` on test data — it would create a new vocabulary and cause data leakage.

Topic 8: Project — Emotion Detection

Q36. Explain the Emotion Detection project step by step.

Answer:

Dataset: Kaggle "Emotions Dataset for NLP" — text;emotion format

Steps:

1. **Import libraries** — numpy, pandas, matplotlib, seaborn, string, nltk
2. **Load data** —

```
pd.read_csv('train.txt', sep=';', header=None, names=['text', 'emotion'])
```
3. **Check null values** — `df.isnull().sum()` → 0 nulls
4. **Convert emotions to numbers** — Create mapping dictionary (sadness=0, anger=1, love=2, etc.)
5. **Text preprocessing:**
 - Lower casing
 - Remove punctuations
 - Remove numbers
 - Remove URLs
 - Remove HTML tags
 - Remove emojis (ASCII filter)
 - Remove stopwords (NLTK)
6. **Train-test split** — 80:20
7. **Vectorization** — BoW and TF-IDF
8. **Model training:**
 - Multinomial NB + BoW → 76%
 - Multinomial NB + TF-IDF → 66%
 - Logistic Regression + TF-IDF → **86%** ✓
9. **Evaluation** — `accuracy_score(y_test, y_pred)`
10. **Deployment** — Streamlit UI

Q37. How do you convert categorical emotions to numbers?

Answer:

```
python

unique_emotions = df['emotion'].unique()
# ['sadness', 'anger', 'love', 'surprise', 'fear', 'joy']

emotion_numbers = {}
i = 0
for emotion in unique_emotions:
    emotion_numbers[emotion] = i
    i += 1

df['emotion'] = df['emotion'].map(emotion_numbers)
```

Result: sadness=0, anger=1, love=2, surprise=3, fear=4, joy=5

Q38. What is the difference between `apply()` and `map()` in pandas?

Answer:

<code>apply()</code>	<code>map()</code>
Works on functions	Works on dictionaries/Series
Applied to entire column/row	Applied element-wise
<code>df['col'].apply(func)</code>	<code>df['col'].map(dict)</code>
More flexible	Faster for direct mapping

In project: `map()` used for emotion→number conversion, `apply()` used for text cleaning functions.

Q39. What is the role of `train_test_split` ?

Answer:

`train_test_split` divides data into **training and testing sets**.

```
python

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    df['text'], df['emotion'], test_size=0.2, random_state=42
)
```

- **80% training** — model learns from this
- **20% testing** — model is evaluated on this
- `random_state=42` — ensures reproducibility

Why: To evaluate model performance on unseen data.

Q40. How do you evaluate an NLP classification model?

Answer:

Using:

1. Accuracy Score

```
python

from sklearn.metrics import accuracy_score
accuracy_score(y_test, y_pred)
```

2. Classification Report — precision, recall, F1-score

```
python

from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))
```

3. Confusion Matrix — true positives, false positives, etc.

In project: Accuracy score was used — 86% for Logistic Regression + TF-IDF.

Q41. Why can't Machine Learning models detect sarcasm?

Answer:

Because ML models:

- Look at **word frequency** (BoW, TF-IDF)
- Don't understand **context**
- Don't understand **tone**
- Don't understand **word order** deeply

Example:

- *"Staying up all night debugging is exactly what I dreamed of"*
- Words: "dreamed", "exactly" → sound positive
- But actual meaning: **sarcasm** (negative)

ML model would interpret this as **positive feedback**, which is wrong.

DL models (BERT, GPT) can detect sarcasm better because they understand context.

Q42. Why is "I'm fine" difficult for ML models?

Answer:

- "I'm fine" literally means positive
- But if said sadly, it means **negative**
- ML models only see words, not **tone/emotion**
- BoW/TF-IDF give "fine" a positive weight
- Model predicts **positive**, but actual emotion is **sad**

DL models with embeddings and context can detect this better.

Q43. What is the difference between ML-based NLP and DL-based NLP?

Answer:

ML-Based NLP	DL-Based NLP
Uses BoW, TF-IDF, N-Grams	Uses Word2Vec, GloVe, BERT, Transformers
Sparse vectors	Dense vectors
No context capture	Captures context
Can't detect sarcasm	Better at sarcasm
Fast, less data needed	Slow, needs large data
Good for simple tasks	Good for complex tasks
Punctuation removed	Punctuation kept
Stopwords removed	Stopwords may be kept

Q44. What are Word Embeddings? (Brief — DL topic)

Answer:

Word Embeddings are **dense vector representations** of words that capture **semantic meaning and context**.

Examples:

- **Word2Vec** — Google
- **GloVe** — Stanford
- **FastText** — Facebook
- **BERT, GPT** — Transformers

Key difference from BoW/TF-IDF:

- BoW: sparse, no meaning
- Embeddings: dense, captures relationships
 - Example: king - man + woman $\approx$ queen

Covered in: Deep Learning NLP (future videos)

Q45. What is the difference between `fit`, `transform`, and `fit_transform`?

Answer:

Method	What it does	When to use
<code>fit</code>	Learns parameters (vocabulary)	Training data
<code>transform</code>	Applies learned parameters	Test data
<code>fit_transform</code>	Learns + applies	Training data (combined)

Example:

```
python

# Training
X_train_vec = vectorizer.fit_transform(X_train)

# Testing
X_test_vec = vectorizer.transform(X_test)
```

Never use `fit_transform` on test data — it causes data leakage.

Q46. What is data leakage? How to avoid it?

Answer:

Data leakage occurs when **test data information** leaks into the training process.

Example: Using `fit_transform()` on test data — the vectorizer learns vocabulary from test data, which shouldn't happen.

How to avoid:

- Always `fit_transform()` on **training data**

- Always `transform()` on **test data**
- Split data **before** preprocessing/vectorization

Q47. What is the difference between `CountVectorizer` and `TfidfVectorizer`?

Answer:

CountVectorizer	TfidfVectorizer
Counts word frequency	Uses $TF \times IDF$
Values = integers (0,1,2...)	Values = floats (0.0–1.0)
Same weight to all words	Different weights
Simpler	More informative
Used for BoW	Used for TF-IDF
<code>binary=True</code> → 0/1	Always weighted

In project: Both were used — Logistic Regression + TF-IDF gave best results (86%).

Q48. What is `ngram_range` in `CountVectorizer`?

Answer:

`ngram_range` specifies the range of N-grams to use.

python

```
CountVectorizer(ngram_range=(1,1)) # Only unigrams
CountVectorizer(ngram_range=(1,2)) # Unigrams + Bigrams
CountVectorizer(ngram_range=(2,2)) # Only Bigrams
CountVectorizer(ngram_range=(1,3)) # Unigrams + Bigrams + Trigrams
CountVectorizer(ngram_range=(3,3)) # Only Trigrams
```

Why use it:

- Captures word order
- Better semantic meaning
- But increases dimensionality

Q49. What is the role of `random_state` in `train_test_split`?

Answer:

`random_state` ensures **reproducibility** — the same split every time you run the code.

python

```
train_test_split(X, y, test_size=0.2, random_state=42)
```

- Without it, the split is random each time
- With it, the same rows go to train/test every time

- Important for **debugging and comparison**

Q50. How would you improve the Emotion Detection project?

Answer:

1. **Try more models** — SVM, Random Forest, XGBoost
2. **Hyperparameter tuning** — GridSearchCV, RandomizedSearchCV
3. **Try N-Grams** — `ngram_range=(1,2)` or `(1,3)`
4. **Tune TF-IDF parameters** — `max_features`, `min_df`, `max_df`
5. **Use Word Embeddings** — Word2Vec, GloVe (after DL)
6. **Deep Learning models** — LSTM, BERT, Transformers
7. **Build Streamlit UI** for deployment
8. **Handle class imbalance** if present
9. **Cross-validation** for robust evaluation
10. **Ensemble methods** — combine multiple models

Topic 10: Miscellaneous

Q51. What is NLTK? Why is it used?

Answer:

NLTK = Natural Language Toolkit — a leading Python library for working with human language data.

Features:

- Tokenization
- Stopwords removal
- Stemming
- Lemmatization
- POS tagging
- Parsing
- 50+ corpora and lexical resources (WordNet, etc.)

Install:

```
python

import nltk
nltk.download('punkt')
nltk.download('stopwords')
```

Alternative: spaCy (faster, production-ready)

Q52. What is the difference between NLTK and spaCy?

Answer:

NLTK	spaCy
Research-oriented	Production-oriented
Slower	Faster
More features	Optimized features
Educational	Industrial
Many corpora	Pre-trained models
<code>nltk.word_tokenize()</code>	<code>spacy.load('en_core_web_sm')</code>

Both are used for NLP preprocessing.

Q53. What is the difference between Stemming and Lemmatization?

Answer:

Stemming	Lemmatization
Cuts word to root form	Uses dictionary to find base form
Faster, cruder	Slower, accurate
"running" → "run"	"running" → "run"
"better" → "better"	"better" → "good"
No dictionary needed	Needs dictionary (WordNet)
PorterStemmer, SnowballStemmer	WordNetLemmatizer

Both reduce vocabulary size.

Note: Covered in next portion of the course (Stemming, Lemmatization, etc.)

Q54. What is Tokenization? How is it different from splitting?

Answer:

Tokenization is the process of **splitting text into individual tokens** (words, punctuation, symbols).

Difference from splitting:

Splitting	Tokenization
<code>.split()</code> on spaces	<code>word_tokenize()</code> on spaces + punctuation
"I am happy!" → ["I", "am", "happy!"]	"I am happy!" → ["I", "am", "happy", "!"]
Doesn't separate punctuation	Separates punctuation
Simple	More accurate

Example:

python

```
from nltk.tokenize import word_tokenize
word_tokenize("I am happy!")
# → ['I', 'am', 'happy', '!']
```

Q55. What are the pros and cons of Bag of Words?

Answer:

Pros:

-  Easy to implement
-  Fixed size — works with ML models
-  Fast and efficient
-  Works well for simple classification

Cons:

-  Sparsity (many zeros)
-  Out of Vocabulary (OOV)
-  No word order
-  Limited semantic meaning
-  Common words get high weight

Q56. What are the pros and cons of TF-IDF?

Answer:

Pros:

-  Information retrieval (Google Search)
-  Different weights based on importance
-  Better than BoW for many tasks
-  Reduces impact of common words

Cons:

-  Sparsity
-  Out of Vocabulary (OOV)
-  High dimensionality (50,000+)
-  Limited semantic meaning
-  Still can't detect sarcasm

Q57. What is the "Bag" in Bag of Words?

Answer:

"Bag" refers to a **jumbled, unordered collection** of words.

- Grammar is ignored
- Word order is ignored

- Only **word frequency** matters
- Like putting words in a bag and shaking them

Example:

- "I love NLP" and "NLP love I" → same BoW vector

That's why it's called "**Bag**" of Words.

Q58. What is the difference between `isnull()` and `isna()` in pandas?

Answer:

They are **aliases** — both do the same thing.

```
python

df.isnull().sum()  # Same
df.isna().sum()    # Same
```

Usage:

```
python

df.isnull().sum()  # Count nulls per column
df.isnull().sum().sum() # Total nulls
```

In project: `df.isnull().sum()` showed 0 nulls.

Q59. What is the difference between `loc` and `iloc` in pandas?

Answer:

<code>loc</code>	<code>iloc</code>
Label-based	Integer position-based
<code>df.loc[0]</code>	<code>df.iloc[0]</code>
<code>df.loc['row', 'col']</code>	<code>df.iloc[0, 1]</code>
Includes end label	Excludes end index

In project: `df.loc[0]` used to select first row.

Q60. How do you handle Out of Vocabulary (OOV) in practice?

Answer:

1. **Use <UNK> token** — replace unknown words with a special token
2. **Subword tokenization** — BPE, WordPiece (BERT), SentencePiece
3. **Character-level models** — fallback to characters
4. **Use pre-trained embeddings** — Word2Vec, GloVe, FastText
5. **Increase vocabulary size** — `max_features=None`

6. Use DL models — BERT, GPT handle OOV via subwords

In ML-based NLP: OOV is a persistent problem; DL handles it better.

Quick Revision Table

Topic	Key Points
NLP	AI subfield, bridges human & computer language
Approaches	Rule-Based → ML → DL
Pipeline	Gather → Clean → Vectorize → Train → Evaluate
Cleaning	Lowercase, punctuation, numbers, URLs, HTML, emojis, stopwords
Vectorization	One-Hot → BoW → N-Grams → TF-IDF
BoW	Word frequency, fixed size, no order
N-Grams	Groups of N words, better semantics
TF-IDF	$TF \times IDF$, weighted, information retrieval
Models	Multinomial NB, Logistic Regression, SVM
Best Result	Logistic Regression + TF-IDF → 86%
DL NLP	Word2Vec, GloVe, BERT, Transformers

End of Interview Questions

💡 **Tip:** Practice explaining these answers in your own words. Focus on **why** behind each concept, not just **what**.